

Redes Programables y el Lenguaje P4: Del Plano de Datos Fijo al Plano de Datos Programable

Christopher Zúñiga (C28730) Adrian Hernandez (C39321)

Curso de Redes de Computadoras

Investigación #2 — Redes Programables con P4

Junio de 2026

Repositorio: <https://github.com/czuniga28/redes-programables-p4>

Abstract

Las redes tradicionales se construyeron sobre dispositivos cuyo plano de datos era fijo: el conjunto de protocolos y operaciones que un switch o router podía ejecutar venía determinado por el fabricante en silicio. Este modelo limitó la velocidad de innovación y obligó a los operadores a esperar ciclos de hardware de años para soportar nuevos encabezados o funciones. El surgimiento del plano de datos programable, y en particular del lenguaje P4 (*Programming Protocol-Independent Packet Processors*), invirtió esta relación: el ingeniero define cómo se parsea, procesa y reenvía cada paquete, y compila ese comportamiento hacia objetivos de software (BMv2), ASIC (Tofino) o FPGA. Este informe revisa la evolución del plano de datos, la arquitectura y el modelo de ejecución de P4 (PISA), el protocolo P4Runtime para la comunicación con el plano de control vía gRPC, y casos de uso representativos como la telemetría in-band (INT), el balanceo de carga programable y las funciones de seguridad en el plano de datos, incluyendo despliegues de producción en hiperscaladores.

Index Terms

Redes programables, P4, SDN, plano de datos, PISA, P4Runtime, BMv2, INT.

I. INTRODUCCIÓN

Durante décadas, la arquitectura de los equipos de red siguió un patrón *bottom-up*: el hardware fijaba qué protocolos existían y el software de control se adaptaba a esas capacidades. Un switch Ethernet “sabía” procesar VLAN, IPv4 o MPLS porque su circuito integrado de aplicación específica (ASIC) había sido diseñado para ello. Añadir un nuevo encabezado —por ejemplo, un formato de encapsulamiento para centros de datos— implicaba un nuevo *tape out* de silicio, un proceso de varios años y altísimo costo [2].

La programabilidad del plano de datos propone el patrón inverso, *top-down*: el operador describe en un lenguaje de alto nivel cómo deben tratarse los paquetes, y un compilador traduce esa descripción a una pipeline configurable. P4 es hoy el lenguaje de facto para expresar ese comportamiento [1]. Este trabajo construye el marco conceptual necesario para comprender la arquitectura del plano de datos programable, como base para las implementaciones de laboratorio (un router IPv4 estático y un sistema de contadores de tráfico) que acompañan a esta investigación.

II. EVOLUCIÓN DEL PLANO DE DATOS

A. Limitaciones del modelo de red tradicional

En el modelo clásico, el plano de datos es *fixed-function*: el conjunto de campos reconocibles, las tablas de coincidencia y las acciones disponibles están cableados. Esto acarrea tres limitaciones fundamentales:

- **Rigidez de protocolos.** Soportar un nuevo encabezado requiere hardware nuevo. Los operadores quedan atados al catálogo de protocolos que el fabricante decidió implementar.
- **Innovación lenta.** El ciclo de diseño de un ASIC de red ronda los 2–3 años, lo que desacopla la velocidad de la investigación de redes de la velocidad del despliegue real [2].
- **Opacidad y dependencia del proveedor.** El comportamiento exacto del *forwarding* es una caja negra; el operador no puede auditar ni modificar cómo se procesa internamente un paquete.

B. Surgimiento del plano de datos programable

Dos avances habilitaron el cambio. Primero, la *Software-Defined Networking* (SDN) separó el plano de control del plano de datos, centralizando la lógica de decisión en un controlador [3]. Segundo, el trabajo sobre *Reconfigurable Match Tables* (RMT) demostró que era posible construir un ASIC con una pipeline de *match-action* reconfigurable a velocidad de línea (terabits por segundo) sin sacrificar rendimiento [2]. RMT probó que “programable” y “rápido” no eran mutuamente excluyentes, abriendo la puerta a que el plano de datos —no solo el de control— fuera definido por software.

TABLE I
OPENFLOW (SDN CLÁSICA) vs. P4

Aspecto	OpenFlow	P4
Plano de datos	Fijo (campos pre-definidos)	Programable (parser definido por el usuario)
Protocolos	Catálogo cerrado, crece por versión	Independiente del protocolo
Rol del estándar	Protocolo control-switch	Lenguaje para describir el data plane
Pipeline	Tablas con semántica fija	Match-action definidas por el programa
Control	Controlador OpenFlow	P4Runtime / gRPC

C. SDN clásica (OpenFlow) frente a P4

OpenFlow fue el primer protocolo SDN ampliamente adoptado, pero opera sobre un plano de datos *fijo*: define un conjunto cerrado y creciente de campos de coincidencia (*match fields*) que el switch debe entender de antemano. Cada nueva versión de OpenFlow añadía campos (de 12 en la v1.0 a más de 40 en versiones posteriores), evidenciando que un enfoque “enumerar todos los protocolos posibles” no escala [1].

P4 ataca el problema en la raíz: en lugar de fijar los protocolos, deja que el programador los *declare*. La Tabla I resume las diferencias.

Es importante notar que P4 y SDN no son excluyentes: P4 describe *qué* hace el switch, mientras que un protocolo de control (P4Runtime) cumple el rol que OpenFlow tenía de poblar las tablas en tiempo de ejecución.

III. EL LENGUAJE P4

A. Historia y motivación

P4 fue presentado formalmente por Bosshart *et al.* en 2014 en *ACM SIGCOMM Computer Communication Review* [1]. El artículo planteaba tres objetivos de diseño que siguen vigentes:

- 1) **Reconfigurabilidad**: el controlador debe poder redefinir el parsing y el procesamiento de paquetes en el campo.
- 2) **Independencia de protocolo**: el switch no debe estar atado a protocolos específicos; estos se describen en el programa.
- 3) **Independencia del objetivo (*target*)**: el mismo programa debe poder compilarse a distintos dispositivos sin que el programador conozca los detalles del hardware.

La motivación surgió directamente de las limitaciones de OpenFlow y del trabajo RMT: si el hardware ya podía ser reconfigurable, hacía falta un lenguaje de alto nivel, independiente del fabricante, para expresar esa configuración [1], [2].

B. Arquitectura de un programa P4

Un programa P4 (en su versión $P4_{16}$ [4]) se organiza en bloques funcionales que reflejan el recorrido de un paquete por la pipeline:

- **Parsers**. Una máquina de estados que extrae los encabezados del flujo de bits entrante. Cada estado lee un encabezado y decide la transición según un campo (p.ej., el `etherType` para pasar de Ethernet a IPv4).
- **Match-action tables**. El corazón del modelo. Cada tabla define una *clave* (campos a coincidir, con semántica *exact*, *lpm* o *ternary*) y un conjunto de *acciones* posibles. El plano de control puebla las entradas; el plano de datos, ante cada paquete, busca la entrada coincidente y ejecuta su acción (p.ej., decrementar el TTL y fijar el puerto de salida).
- **Control flows**. La lógica imperativa (bloques `apply`) que encadena la aplicación de tablas y condicionales, definiendo el orden del procesamiento en ingreso y egreso.
- **Deparsers**. El proceso inverso al parser: reensambla los encabezados (posiblemente modificados) de vuelta en un flujo de bits para transmitir el paquete.

En el laboratorio asociado, el router IPv4 ejemplifica este flujo: un parser para Ethernet/IPv4, una tabla `ipv4_lpm` con coincidencia *Longest Prefix Match*, acciones que reescriben las MAC y decrementan el TTL, y un deparser que

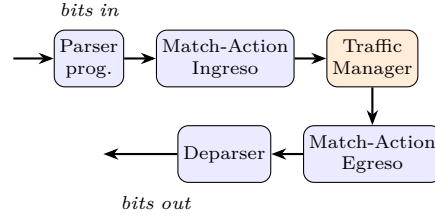


Fig. 1. Pipeline abstracto PISA: parser programable, etapas de match-action en ingreso, *traffic manager*, match-action en egreso y deparser. Ninguna etapa conoce protocolos de antemano; los define el programa P4.

recompone la trama. El Listado 1 muestra, en $P4_{16}$, la declaración de esa tabla: nótese que el programa describe *qué* campo se coincide y *qué* acciones existen, pero las entradas concretas las instala el plano de control.

Listing 1. Tabla de enrutamiento LPM en P4 (extracto del router del laboratorio).

```

action ipv4_forward(macAddr_t dstAddr, egressSpec_t port) {
    standard_metadata.egress_spec = port;
    hdr.ethernet.dstAddr = dstAddr;    // MAC del siguiente salto
    hdr.ipv4.ttl = hdr.ipv4.ttl - 1;   // decremento de TTL
}
table ipv4_lpm {
    key      = { hdr.ipv4.dstAddr: lpm; }
    actions  = { ipv4_forward; drop; NoAction; }
    size     = 1024;
    default_action = drop();           // sin ruta => descarte
}

```

C. Modelo de ejecución PISA

P4 asume un modelo abstracto de máquina conocido como *PISA* (Protocol Independent Switch Architecture). En PISA, el paquete atraviesa secuencialmente: (1) un *parser* programable; (2) una serie de etapas de *match-action* en el ingreso; (3) un *traffic manager* que encola y replica paquetes; (4) etapas de *match-action* en el egreso; y (5) un *deparser* [2], [4]. Cada etapa dispone de memoria (TCAM/SRAM) para tablas y de unidades aritmético-lógicas para las acciones, todo ejecutándose a velocidad de línea. La clave de PISA es que esta estructura es *independiente del protocolo*: no hay nada intrínsecamente “IPv4” o “Ethernet” en el hardware; esos conceptos los introduce el programa P4.

La Fig. 1 esquematiza este recorrido.

La arquitectura concreta se expone al programador mediante un *architecture model*. El más común para aprendizaje es *v1model* (usado por BMv2), que define los bloques *parser*, *ingress*, *egress*, *deparser* y los *externs* disponibles (contadores, registros, *meters*, funciones de *hash*). Arquitecturas como PSA (Portable Switch Architecture) o TNA (Tofino Native Architecture) ofrecen modelos más ricos para hardware real.

D. Tipos de targets

Un mismo programa P4 puede compilarse a objetivos muy distintos:

- **BMv2 (software)**. El *behavioral model* versión 2 es un switch P4 de referencia que corre en CPU; no es de alto rendimiento, pero es ideal para desarrollo, pruebas y docencia [6]. Es el objetivo usado en este laboratorio junto con Mininet.
- **Tofino (ASIC)**. La familia Intel/Barefoot Tofino implementa PISA en silicio y procesa a varios Tbps, siendo el principal objetivo de hardware de producción para P4 [2].
- **FPGAs y SmartNICs**. Plataformas como NetFPGA o las SmartNIC con soporte P4 permiten desplegar pipelines programables cerca del servidor, útiles para *offloading* y funciones de red virtualizadas.

Esta portabilidad —escribir una vez, compilar a múltiples objetivos— es uno de los argumentos centrales a favor de P4.

E. Primitivos de estado: contadores, registers y meters

Más allá del *forwarding*, P4 expone *externs* que permiten mantener estado en el plano de datos, esenciales para monitoreo y telemetría. La Tabla II los compara. Estos primitivos son la base del segundo laboratorio de esta investigación, que mide tráfico por flujo y por protocolo.

TABLE II
PRIMITIVOS DE ESTADO EN P4 (*v1model*)

Primitivo	Qué hace	Uso típico
counter	Cuenta paquetes y/o bytes en celdas indexadas	Estadísticas por protocolo, por puerto
direct_counter	Contador ligado 1:1 a las entradas de una tabla	Conteo por flujo / por regla
register	Memoria de lectura/escritura arbitraria por índice	Estado por flujo, detección de elefantes
meter	Marca paquetes según tasa (RFC 2698)	Limitación de tasa, QoS

Mientras que un **counter** solo incrementa, un **register** permite *leer-modificar-escribir* dentro del pipeline, habilitando algoritmos con estado a velocidad de línea (p. ej., acumular los bytes de un flujo y compararlos contra un umbral para marcarlo como “elefante”). Los *direct counters*, asociados a entradas de tabla, son la forma natural de contabilizar por flujo sin gestionar índices manualmente. La distinción es importante: P4 no solo decide *a dónde* va un paquete, sino que puede *recordar* información entre paquetes, base de la telemetría y la seguridad en el data plane.

IV. P4RUNTIME Y EL PLANO DE CONTROL

A. Rol del plano de control

Un programa P4 define la *estructura* del plano de datos (qué tablas existen, qué acciones soportan), pero no su *contenido*: las entradas concretas de las tablas las instala el plano de control en tiempo de ejecución. En el router del laboratorio, por ejemplo, el programa declara la tabla `ipv4_lpm`, pero las rutas estáticas (prefijo → siguiente salto, puerto) las inserta el control plane. Esta separación es la misma idea de SDN, ahora extendida a un data plane definido por el usuario.

B. P4Runtime y gRPC

P4Runtime es la API estándar, independiente del objetivo, para que un controlador gestione un dispositivo P4 en ejecución [5]. Sus características principales:

- Se basa en **gRPC** y *Protocol Buffers*, lo que le da eficiencia, *streaming* bidireccional y soporte multiplataforma.
- Es **independiente del programa**: el controlador descubre las tablas y acciones a partir del archivo `P4Info` que genera el compilador, en lugar de tener cableados los nombres de campos.
- Permite insertar/leer entradas de tablas, leer contadores y *registers*, y recibir notificaciones (p. ej., paquetes enviados al controlador).

En el laboratorio se emplea, como alternativa didáctica, la interfaz *thrift* de BMv2 (`simple_switch_CLI`) para poblar tablas y leer contadores; conceptualmente cumple el mismo rol que P4Runtime, aunque este último es el camino estándar para producción.

C. Comparación con OpenFlow

P4Runtime y OpenFlow comparten el propósito de comunicar un controlador con el *forwarding plane*, pero difieren en un punto esencial:

- **OpenFlow** asume un pipeline de tablas con semántica fija; el controlador manipula campos que el protocolo ya conoce.
- **P4Runtime** es *agnóstico al esquema*: las tablas, campos y acciones provienen del programa P4 cargado, descritos en `P4Info`. Cambiar el programa P4 cambia automáticamente la “API” que ve el controlador, sin necesidad de un nuevo estándar.

Dicho de otro modo, OpenFlow estandariza *el contenido*; P4 + P4Runtime estandarizan *el mecanismo*, dejando el contenido a cargo del programa [1], [5].

V. CASOS DE USO RELEVANTES

A. In-Band Network Telemetry (INT)

INT es quizá la aplicación insignia de P4. Cada switch programable inserta metadatos en los propios paquetes de datos a su paso —identificador de switch, puerto de entrada/salida, tiempo de encolamiento, marca de tiempo—, de modo que el último salto puede reconstruir la ruta completa y el estado de la red *sin sondeo activo* [7]. Esto habilita una visibilidad por paquete imposible en redes tradicionales, útil para detectar microrráfagas y diagnosticar latencias.

B. Balanceo de carga programable

P4 permite implementar balanceo de carga directamente en el plano de datos, a velocidad de línea. Un ejemplo notable es HULA, que mantiene en *registers* información de congestión por trayecto y selecciona el siguiente salto menos congestionado, logrando balanceo sensible a la carga sin involucrar al controlador en cada decisión [8]. El laboratorio de contadores de esta investigación usa los mismos primitivos (*registers*, contadores) que sustentan estas técnicas.

C. Funciones de seguridad en el plano de datos

Al poder inspeccionar y decidir sobre cada paquete a Tbps, el plano de datos programable se ha convertido en un lugar natural para defensa. Sistemas como Jaqen demuestran detección y mitigación de ataques DDoS volumétricos directamente en switches programables, filtrando tráfico malicioso en el data plane sin saturar servidores intermedios [9]. Otras aplicaciones incluyen *firewalls* con estado, limitación de tasa y *scrubbing* de tráfico.

D. Despliegues de producción

La programabilidad del plano de datos no es solo académica. Un caso emblemático es *SilkRoad* de Microsoft, un balanceador de carga de capa 4 implementado en switches programables que reemplaza grandes parques de balanceadores por software, sosteniendo millones de conexiones en el ASIC y reduciendo drásticamente latencia y costo [10]. En la misma línea, los operadores de hiperescala han adoptado switches Tofino y telemetría basada en INT para obtener visibilidad por paquete en sus *fabrics* de centro de datos, y SmartNICs programables para descargar funciones de red de los servidores [2], [7]. En el ámbito de las telecomunicaciones, P4 se ha aplicado a funciones de *User Plane Function* (UPF) de 5G y a *gateways* celulares, donde el reenvío a Tbps con baja latencia es crítico. La tendencia es clara: el plano de datos deja de ser un componente fijo para convertirse en una plataforma de software sobre la que se despliegan funciones de red completas.

E. Del concepto a la práctica: el laboratorio asociado

Las dos implementaciones que acompañan este informe ejercitan directamente los conceptos anteriores. El *router IPv4 estático* materializa el flujo *parser*→*match-action*→*deparser*: parsea Ethernet/IPv4, consulta una tabla LPM poblada desde el plano de control, decrementa el TTL (descartando en cero y, opcionalmente, respondiendo *ICMP Time Exceeded*), reescribe las direcciones MAC por salto y descarta por defecto el tráfico sin ruta. El *sistema de contadores* ejerce los primitivos de estado de la Tabla II: *direct counters* por flujo (IP origen + IP destino), contadores globales por protocolo y *registers* para acumular bytes y detectar flujos elefante, leídos periódicamente por un controlador en Python sobre la interfaz de control. Así, los mismos mecanismos que sustentan INT o *SilkRoad* en producción se observan, a escala didáctica, sobre BMv2 y Mininet.

VI. CONCLUSIONES

El paso del plano de datos fijo al programable representa un cambio estructural en cómo se construyen las redes. P4, apoyado en el modelo PISA y en hardware reconfigurable como RMT/Tofino, permite que el comportamiento del *forwarding* sea definido por software y portado entre objetivos heterogéneos —de BMv2 en software a ASIC de varios Tbps. P4Runtime completa el cuadro al ofrecer una API de control agnóstica al esquema, superando la rigidez de catálogo de OpenFlow. Los casos de uso —telemetría in-band, balanceo de carga y seguridad en el data plane— muestran que mover lógica al plano de datos habilita capacidades antes inviables. Las implementaciones de laboratorio que acompañan este informe aterrizan estos conceptos: un router IPv4 cuya tabla LPM se programa desde el control plane, y un sistema de contadores que mide tráfico por flujo y protocolo usando los mismos primitivos de P4 que sustentan las aplicaciones de producción.

REFERENCES

- [1] P. Bosshart *et al.*, “P4: Programming Protocol-Independent Packet Processors,” *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.
- [2] P. Bosshart *et al.*, “Forwarding Metamorphosis: Fast Programmable Match-Action Processing in Hardware for SDN,” in *Proc. ACM SIGCOMM*, 2013, pp. 99–110.
- [3] N. McKeown *et al.*, “OpenFlow: Enabling Innovation in Campus Networks,” *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.
- [4] The P4 Language Consortium, “P4₁₆ Language Specification, v1.2.4,” 2023. [Online]. Available: <https://p4.org/p4-spec/>
- [5] The P4.org API Working Group, “P4Runtime Specification,” 2021. [Online]. Available: <https://p4.org/p4runtime/>
- [6] P4 Language Consortium, “Behavioral Model (BMv2) — `simple_switch`,” 2024. [Online]. Available: <https://github.com/p4lang/behavioral-model>
- [7] The P4.org Applications Working Group, “In-band Network Telemetry (INT) Dataplane Specification, v2.1,” 2020. [Online]. Available: <https://p4.org/specs/>
- [8] N. Katta *et al.*, “HULA: Scalable Load Balancing Using Programmable Data Planes,” in *Proc. ACM SIGCOMM Symposium on SDN Research (SOSR)*, 2016.
- [9] Z. Liu *et al.*, “Jaqen: A High-Performance Switch-Native Approach for Detecting and Mitigating Volumetric DDoS Attacks with Programmable Switches,” in *Proc. USENIX Security Symposium*, 2021.
- [10] R. Miao, H. Zeng, C. Kim, J. Lee, and M. Yu, “SilkRoad: Making Stateful Layer-4 Load Balancing Fast and Cheap Using Switching ASICs,” in *Proc. ACM SIGCOMM*, 2017, pp. 15–28.
- [11] B. Lantz, B. Heller, and N. McKeown, “A Network in a Laptop: Rapid Prototyping for Software-Defined Networks,” in *Proc. ACM SIGCOMM HotNets*, 2010.
- [12] P4 Language Consortium, “P4 Tutorials,” 2024. [Online]. Available: <https://github.com/p4lang/tutorials>
- [13] P. Biondi and the Scapy community, “Scapy: Packet crafting for Python,” 2024. [Online]. Available: <https://scapy.net/>